

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: graph_incidentelist.c

Di Mystery Shack, Dipper dan Mabel sedang menghadapi anomali baru. Mereka menemukan catatan rahasia dari Grunkle Ford tentang jaringan portal supernatural yang menghubungkan berbagai titik di Gravity Falls hingga menembus dimensi ke area kampus ITB Jatinangor. Ford yang kini sedang menyelidiki aktivitas paranormal tersebut membutuhkan sistem pelacakan yang efisien untuk mengidentifikasi rute pergerakan entitas gaib dan hubungan antar lokasi yang mencurigakan.

Dipper, yang selalu analitis dalam memecahkan misteri, menyarankan untuk memodelkan jaringan lokasi-lokasi tersebut dalam bentuk **Graph**, di mana setiap titik lokasi adalah *node* dan jalur portal antar lokasi adalah *edge*. Untuk efisiensi ruang halaman di dalam Jurnalnya dan kemudahan penambahan rute baru, Dipper mengusulkan menggunakan representasi **Incidence List**.

Mabel, Soos, dan Wendy pun ikut membantu merekap data. Mereka harus membuat ADT Graph menggunakan incidence list dalam bahasa C. Graph ini bersifat **tidak berarah (undirected)** karena portal supernatural tersebut dapat dilewati bolak-balik. Mereka juga harus menyediakan fungsi untuk menambahkan node (lokasi), menambahkan edge (rute portal), mencetak semua koneksi yang ada, dan mencari tahu apakah dua lokasi terhubung secara langsung.

Namun, mereka menghadapi beberapa tantangan anomali, bagaimana menyimpan daftar koneksi dengan efisien, bagaimana memastikan tidak ada edge ganda (duplikat portal yang bisa membuat mereka tersesat di dimensi paralel), dan bagaimana menghindari *memory leak* (tersedotnya memori ke dalam pusaran ruang-waktu) saat node-node dihapus.

Bantulah Dipper dan kawan-kawan untuk memecahkan misteri implementasi ini dengan melengkapi file `graph_incidentelist.c`. Header file yang digunakan adalah `graph_incidentelist.h`.

Catatan: Graph harus diimplementasikan menggunakan incidence list, bukan incidence matrix.

C

 [graph_incidentelist.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.59 MB
2	10	Accepted	0.00 sec, 1.61 MB
3	10	Accepted	0.00 sec, 1.67 MB
4	10	Accepted	0.00 sec, 1.65 MB
5	10	Accepted	0.00 sec, 1.60 MB
6	10	Accepted	0.00 sec, 1.64 MB
7	10	Accepted	0.00 sec, 1.64 MB
8	10	Accepted	0.00 sec, 1.51 MB
9	10	Accepted	0.00 sec, 1.62 MB
10	10	Accepted	0.00 sec, 1.65 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: graphadjlist.c

Di Nangor Falls, titik-titik anomali mulai saling terhubung oleh portal satu arah. Jika ada portal dari titik **a** ke titik **b**, maka titik **b** dapat dicapai langsung dari titik **a**, tetapi belum tentu sebaliknya.

Soal ini adalah akhir dari rangkaian Pra-Praktikum Algoritma dan Pemrograman semester ini ;) Sebelum tirai ditutup, Dipper dan Mabel masih punya satu tugas terakhir: memetakan seluruh portal anomali agar perjalanan terakhir di Nangor Falls tidak berakhir tersesat di titik yang salah.

Agar jalur portal ini dapat dipantau, Anda diminta membangun **ADT Graph Berarah dengan representasi adjacency list berbentuk multilist sesuai materi slide Graph di perkuliahan**.

Setiap simpul graph disimpan pada *leader list*. Setiap simpul menyimpan **id**, **nPred** yaitu jumlah busur masuk ke simpul tersebut, **trail** yaitu list successor, dan **next** yaitu simpul berikutnya. Setiap elemen pada list successor menyimpan pointer ke simpul tujuan dan pointer ke successor berikutnya.

Tugas Anda pada sesi Pra-Praktikum ini adalah **mengimplementasikan seluruh primitif ADT Graph Berarah dari materi slide Graph di perkuliahan** yang ada pada [graphadjlist.h](#) ke dalam **graphadjlist.c**.

Fungsi dan prosedur yang perlu diimplementasikan adalah:

```
void CreateGraph(int x, Graph *g);

AddrNode newGraphNode(int x);
void deallocGraphNode(AddrNode p);

AddrSuccNode newSuccNode(AddrNode pn);
void deallocSuccNode(AddrSuccNode p);

AddrNode searchNode(Graph g, int x);
AddrSuccNode searchEdge(Graph g, int prec, int succ);

void insertNode(Graph *g, int x, AddrNode *pn);
void insertEdge(Graph *g, int prec, int succ);
void deleteNode(Graph *g, int x);
```

Ketentuan Implementasi:

- **insertNode** menyisipkan simpul pada leader list secara terurut membesar berdasarkan **id**.
- **insertEdge** akan menambahkan simpul **prec** dan/atau **succ** terlebih dahulu jika simpul tersebut belum ada.
- List successor juga disimpan terurut membesar berdasarkan **id** simpul successor.
- Jika busur **<prec, succ>** sudah ada, graph tidak berubah.
- Setiap penambahan busur baru harus menaikkan **nPred** milik simpul tujuan.
- **deleteNode** menghapus simpul tersebut, semua busur keluar dari simpul tersebut, dan semua busur masuk menuju simpul tersebut.
- Setiap node dan successor node yang dihapus harus dikembalikan ke sistem menggunakan **free()**.

Format Masukan & Keluaran:

Sistem/Driver akan menguji ADT Anda secara otomatis dengan membuat graph, menambahkan simpul, menambahkan busur, mencari simpul dan busur, menghapus simpul, serta memeriksa struktur akhir graph.

Kumpulkan implementasi Anda dalam file **graphadjlist.c**!

Catatan:

- **ADT ini akan digunakan kembali pada sesi Praktikum**, sehingga usahakan untuk mendapat nilai sempurna pada soal ini.
- Fungsi dan prosedur ADT tidak perlu mencetak apa pun.

C

 [graphadjlist.c](#)

Score: 100

Blackbox

Score: 100